# Efficient Scheduling for Batched AI Jobs: Minimising Makespan through Model Grouping, Johnson's Rule, and Checkpoint Prefetching

*Jingsheng*


4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2026

# Abstract

Large Language Model (LLM) batch workloads—model benchmarking, dataset labelling, synthetic data generation—submit hundreds of tasks that share common model weights. Existing serverless LLM schedulers treat tasks as independent, causing excessive model switching that dominates execution time.

This dissertation formalises the batch scheduling problem as a two-machine flow-shop (I/O and GPU computation) and introduces three composable optimisations: (1) **model grouping** reduces model switches from $O(n)$ to $O(k)$; (2) **Johnson's Rule ordering** sequences model groups to maximise I/O-computation overlap; and (3) **checkpoint prefetching** preloads the next model's weights into CPU pinned memory while the current group executes.

Evaluation on NVIDIA RTX A5000 GPUs shows that model grouping reduces makespan by 98% over FIFO (5153s to 102.6s for 100 tasks with 2 models). Checkpoint prefetching hides 68.7% of model loading latency. Johnson's Rule provides 7.0–21.8% additional makespan reduction over naive orderings. The shared-aware strategy outperforms FIFO by 5.3–86$\times$ across batch sizes 10–200 and model sizes 0.6B–32B, with advantages growing as batch size increases. Prefetching remains effective across storage tiers including 10 GbE network storage (78.9% latency hidden), making remote storage viable for production deployments.

# Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

# Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Jingsheng*)

# Acknowledgements

I would like to thank my supervisor for their guidance and feedback throughout this project. I am grateful to the ServerlessLLM team for providing the open-source framework that made this work possible, and to the School of Informatics for providing access to the GPU cluster used for experimental evaluation.

# Table of Contents

# Chapter 1

# Introduction

The emergence of Large Language Models (LLMs) has created substantial demand for batch inference workloads. Unlike online serving where individual requests require low-latency responses, batch workloads submit thousands of tasks together for offline processing. Common scenarios include:

- **Model Benchmarking**: Evaluating multiple candidate models on the same prompt set (e.g., MMLU, HumanEval, MT-Bench) to generate scoring matrices.
- **Dataset Processing**: Using a single model to label, translate, or summarise large corpora for downstream training.
- **Synthetic Data Generation**: Creating training data by applying the same model and prompt template to numerous data points.

These batch workloads exhibit a critical property that existing schedulers ignore: *tasks within a batch are not independent*. They share substantial computational structure—identical model weights and common prompt prefixes—that can be exploited to reduce total execution time.

## 1.1 Motivation

Consider a typical model benchmarking scenario: evaluating two LLMs (Model A and Model B) on 10 questions each, totalling 20 tasks. A naive scheduler might execute tasks in submission order: ABABABAB... This alternating pattern forces the system to repeatedly load and unload model weights from GPU memory. Our experiments show that each model switch incurs approximately 108 seconds of overhead. With 10 switches, the cumulative switching time is 1080 seconds—dominating the total execution time of 1119.85 seconds. In contrast, grouping tasks by model (AAAA...BBBB) requires only 1 switch, completing in 146.56 seconds—an 87% reduction.

Figure 1.1 illustrates this difference. The insight is straightforward: *batch tasks contain exploitable shared structure, and the scheduler should discover and leverage this structure to minimise makespan*. Reducing model switches from $O(n)$ to $O(k)$ through grouping provides an order-of-magnitude improvement.

However, grouping alone does not address the remaining $O(k)$ model switches between groups. Each switch still incurs a full model load (e.g., 60 GB for a 32B model, taking $>$20 s from NVMe SSD). **Checkpoint prefetching** hides this latency by loading the next model's weights into CPU pinned memory while the current group executes on GPU. Since I/O and GPU computation use different hardware resources, they can proceed in parallel.

Prefetching effectiveness depends on the *order* in which groups execute: a group with long compute time provides a large overlap window, while a short group provides little. For a three-model batch (0.6B, 8B, 32B), size-descending order (32B→8B→0.6B) wastes 21.7 s of GPU-idle loading at the start, yielding 81.7 s makespan. Johnson's Rule ordering (0.6B→8B→32B) starts with a tiny 0.4 s load, uses each group's compute time to hide the next model's prefetch, and achieves 67.1 s—**18% faster**. This pattern—two sequential machines (I/O and GPU), each job visits both, minimise completion time—is precisely the *two-machine flow-shop* problem Johnson (1954). Johnson's Rule provides the provably optimal ordering.

The three components compose into a complete pipeline: grouping reduces switches from $O(n)$ to $O(k)$, prefetching hides I/O behind computation, and Johnson's Rule maximises overlap. Despite this opportunity, existing serverless LLM schedulers treat batch tasks as independent, resulting in excessive model switching and no I/O-computation overlap.

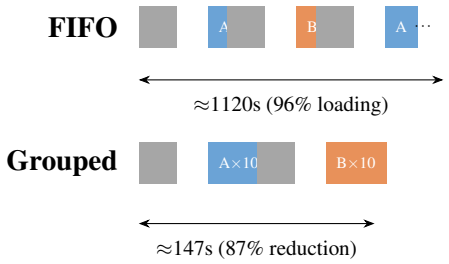


Figure 1.1: FIFO vs Model-Grouped scheduling for 20 tasks with 2 models. FIFO incurs 10 switches (each ∼108s); grouping requires only 1 switch.

## 1.2 Problem Definition

We formalise the batch scheduling problem as follows:

**Input:**

- A batch of tasks $B = \{t_1, t_2, ..., t_n\}$
- Each task $t_i = (m_i, p_i, \theta_i)$ where $m_i$ is the model, $p_i$ is the prompt, and $\theta_i$ are generation parameters
- A cluster of $k$ GPU workers $W = \{w_1, w_2, ..., w_k\}$
- Each worker can load at most one model at a time, with loading latency $L_{load}(m)$
- Task inference latency is $L_{infer}(t_i)$

**Sharing Relationships:**

- **Model Sharing**: If $m_i = m_j$, tasks $t_i$ and $t_j$ can share loaded model weights, avoiding redundant loading. This is the primary sharing relationship exploited in this work.
- **Prefix Sharing**: If $\text{prefix}(p_i) = \text{prefix}(p_j)$, tasks can benefit from inference engine cache reuse. This is a secondary sharing relationship identified but not exploited in this work (see Chapter 7).

**Optimisation Objective:**

$$\text{Minimise } C_{max} = \max_{i \in \{1,\ldots,n\}} \text{completion\_time}(t_i)$$

That is, minimise the *makespan*—the time when the last task completes.

**Constraints:**

- Each worker loads at most one model at a time (single-model constraint)
- Model switching incurs $L_{load}$ overhead (cold start penalty)
- Worker count $k$ can be adjusted via auto-scaling

## 1.3 Research Questions

This dissertation addresses three research questions:

**RQ1:** How can we discover shared structure (shared models) in a batch of tasks and exploit it to reduce makespan? Specifically, how does model grouping compare to naive FIFO scheduling?

**RQ2:** Can the batch scheduling problem be modelled as a two-machine flow-shop (I/O and compute), and does Johnson's Rule provide an effective ordering of model groups to maximise I/O-computation overlap via checkpoint prefetching?

**RQ3:** How do different scheduling strategies (FIFO, Random, Model Grouping, Shared-Aware with prefetching) compare across varying batch sizes, model sizes, and storage tiers (local SSD, RAID, network storage)?

## 1.4 Contributions

This work makes the following contributions:

1. **Two-Machine Flow-Shop Formulation**: We model the batch scheduling problem as a classical two-machine flow-shop where Machine A is I/O (loading model weights from SSD to CPU pinned memory) and Machine B is GPU computation, enabling the application of Johnson's Rule for provably optimal model group ordering (Experiment 3: 7.0–21.8% improvement over naive orderings).

2. **Shared-Aware Scheduling Algorithm**: An algorithm that analyses batch structure to discover model sharing relationships, groups tasks by model to minimise switches from $O(n)$ to $O(k)$, and applies Johnson's Rule to order groups for maximal I/O-computation overlap (Experiment 1: 50× speedup over FIFO).
3. **Checkpoint Prefetching Mechanism**: A prefetching policy that leverages ServerlessLLM's sllm-store pinned memory pool to begin loading the next model group's weights while the current group executes, hiding I/O latency through pipelining (Experiment 2: 68.7% latency hidden).
4. **Network-Aware Prefetching Extension**: Design analysis and evaluation of prefetching across storage tiers including RAID-0, 10 GbE NFS, and 100 GbE NFS, demonstrating that the prefetch architecture generalises to network-attached replicated storage (Experiment 5: 78.9% latency hiding for 10 GbE).
5. **Multi-GPU Load Balancing and Batch API**: A two-level scheduling strategy combining greedy LPT assignment across GPUs with per-GPU Johnson's Rule ordering, and an OpenAI-compatible batch REST API extended for multi-model batches.
6. **Systematic Experimental Evaluation**: Five experiments comparing FIFO, Random, Model Grouping, and Shared-Aware strategies across batch sizes (10–200), model sizes (0.6B–32B), and storage tiers (SSD, RAID, NFS).

In total, the project involved implementing approximately 2,000 lines of new Python code on top of the existing ServerlessLLM framework (∼15K LoC).

## 1.5 Scope and Boundaries

This work focuses strictly on *cluster-level scheduling*. We do not modify or optimise the inference engine itself. Specifically, we do not address:

- KV cache management within the inference engine (e.g., PagedAttention, block allocation)
- Prefill-decode separation or continuous batching internals
- GPU kernel optimisations or quantization techniques

Our scheduler operates at a higher level: it decides *which tasks to execute, in what order, on which workers*. The inference engine (vLLM) handles the actual computation. This separation of concerns is deliberate—scheduler-level optimisations are complementary to engine-level optimisations and can be applied without modifying the inference engine.

# Chapter 2

# Background

This chapter introduces the technical foundations required to understand the shared-aware scheduling system. We cover the transformer architecture and its inference characteristics, the KV cache mechanism and prefix caching, GPU memory hierarchy and pinned memory transfers, serverless computing and the cold start problem, storage hierarchy (local SSD, RAID, network-attached storage), and the ServerlessLLM framework that this work extends.

## 2.1 Transformer-Based LLM Inference

Large Language Models are built on the transformer architecture (Vaswani et al., 2017), which generates output tokens autoregressively. Inference consists of a compute-bound *prefill phase* ($O(L^2)$ attention over the input prompt) followed by a memory-bandwidth-bound *decode phase* (one token per step, each reading the full model weights). Modern LLMs range from 0.5B to 405B parameters (Brown et al., 2020; Touvron et al., 2023); an 8B model requires $\sim$15 GB in FP16 precision, while a 70B model requires $\sim$140 GB — exceeding the memory of a single GPU. Loading these weights from storage to GPU memory is a major bottleneck, taking 80–120 seconds for an 8B model from SSD.

**Continuous batching.** To improve GPU utilisation, vLLM (Kwon et al., 2023) introduced *continuous batching* (also called iteration-level scheduling): rather than waiting for an entire request batch to finish before admitting new requests, the inference engine inserts new requests into the batch at every decode step as slots become free. This keeps the GPU near full utilisation during single-model serving. However, continuous batching operates *within* a single loaded model; it does not address the problem of switching between models in a multi-model batch.

**Tensor parallelism.** When a model's weights exceed a single GPU's VRAM, *tensor parallelism* (Shoeybi et al., 2019) shards weight matrices across multiple GPUs, with each GPU computing a slice of each matrix multiplication and exchanging activations via all-reduce. This enables inference on 70B+ models using multiple GPUs. Our work assumes each model fits within a single GPU's VRAM; tensor parallelism is an orthogonal technique that could be combined with our scheduler when models span

multiple GPUs.

## 2.2 KV Cache and Prefix Caching

During autoregressive generation, inference engines cache key and value tensors (the *KV cache*) to avoid redundant computation. vLLM (Kwon et al., 2023) manages KV cache in fixed-size blocks via *PagedAttention*, eliminating fragmentation. When multiple requests share a common prompt prefix, the KV blocks can be computed once and reused (Zheng et al., 2024). However, cache effectiveness depends on task ordering: interleaving different prefixes causes eviction. This prefix-level optimisation is complementary to our model grouping but is not directly exploited in this work (see Chapter 7).

## 2.3 GPU Memory Hierarchy and Pinned Memory

Loading model weights from storage to GPU involves a multi-level memory hierarchy:

1. **Disk/SSD → CPU memory** (host RAM): Read checkpoint files via filesystem I/O. Bandwidth: 2–5 GB/s (NVMe SSD).
2. **CPU memory → GPU memory** (VRAM): Transfer via PCIe DMA. Bandwidth: 16–32 GB/s (PCIe Gen4 x16).

The CPU→GPU transfer step is significantly faster when using *pinned memory* (also called page-locked memory). Regular (pageable) memory may be swapped to disk by the OS, requiring an extra copy to a pinned staging buffer before DMA transfer. Pinned memory, allocated via `cudaHostAlloc` or registered via `cudaHostRegister` (NVIDIA, 2024a), is guaranteed to remain in physical RAM, enabling direct DMA transfers without intermediate copies.

ServerlessLLM's sllm-store component (Fu et al., 2024) exploits this by pre-allocating a pinned memory pool at startup. Model weights are loaded from SSD directly into this pool using multi-threaded I/O, then transferred to GPU via DMA. This two-stage pipeline (SSD→pinned CPU→GPU) achieves near-peak PCIe bandwidth, reducing model loading time compared to naive `torch.load()` which uses pageable memory.

## 2.4 Serverless Computing and Cold Starts

Serverless functions suffer from *cold starts*: the latency of initialising a new instance (Shahrad et al., 2020). For LLM inference, cold starts are particularly severe — loading model weights takes 80–120 s for an 8B model, which is 100–200× larger than per-request inference latency (∼0.5 s). ServerlessLLM (Fu et al., 2024) mitigates this through locality-enhanced scheduling that places inference on nodes with locally cached checkpoints.

## 2.5 ServerlessLLM Architecture

ServerlessLLM (Fu et al., 2024) is an open-source serverless LLM inference framework designed for low-latency model loading. Its architecture consists of four components:

- **Head Node**: Receives inference requests via an API gateway, maintains deployment state in a SQLite database, and dispatches tasks to workers via a Router.
- **Workers (Pylets)**: GPU nodes managed by the Pylet process manager. Each worker runs vLLM (Kwon et al., 2023) instances for inference. Pylet handles instance lifecycle (start, stop, health checks) and reports resource availability.
- **sllm-store**: A C++ storage daemon running on each worker that manages a pinned memory pool and provides gRPC endpoints for model loading. Key operations include `LoadModelAsync` (load weights into pinned CPU memory) and `RegisterModel` (register a model path). sllm-store uses multi-threaded I/O to saturate SSD bandwidth during loading (gRPC Authors, 2024).
- **AutoScaler**: Monitors deployment state and adjusts the number of vLLM instances based on demand. The default policy is reactive: scale up when queue depth exceeds a threshold, scale down after an idle timeout.

ServerlessLLM's current scheduler is designed for online serving: it assigns incoming requests to any available worker using round-robin. For batch workloads, this has two limitations: (1) tasks are assigned independently without considering shared models or prefixes, leading to excessive model switching; and (2) scaling is reactive rather than proactive, causing unnecessary cold starts at the beginning of a batch.

## 2.6 Storage Hierarchy and Network-Attached Model Repositories

The choice of storage tier directly affects the batch scheduling problem: it determines the I/O time $a_i$ in the two-machine flow-shop formulation (Section 2.7.1), and thus the effectiveness of checkpoint prefetching.

Local NVMe SSDs provide 2–5 GB/s sequential read; RAID-0 ($4\times$) aggregates to 8–12 GB/s. Remote storage — object storage (S3, GCS) or network file systems (NFS, Lustre) — provides unlimited capacity and built-in replication, with bandwidth ranging from 1.2 GB/s (10 GbE) to 12 GB/s (100 GbE). Production systems use local SSD as a cache and remote storage as the source of truth.

Table 2.1 quantifies the scheduling implications.

When storage bandwidth exceeds $\sim$10 GB/s, the I/O stage becomes negligible for medium models and prefetching provides minimal benefit. For slower tiers (10 GbE NFS), loading a 32B model takes $\sim$50 s, requiring the preceding group to provide sufficient compute time for prefetch overlap. Johnson's Rule ordering becomes increasingly important as storage bandwidth decreases.

Table 2.1: Storage tier characteristics and scheduling implications. Load times calculated as checkpoint_size/bandwidth.

| Storage Tier | Read BW | 8B Load | 32B Load | Prefetch? |
|---|---|---|---|---|
| Single NVMe | 3 GB/s | 5.1 s | 20.0 s | Yes (medium+) |
| RAID-0 (4×) | 12 GB/s | 1.3 s | 5.0 s | 32B+ only |
| 10 GbE NFS | 1.2 GB/s | 12.8 s | 50.0 s | Critical |
| 100 GbE NFS | 12.5 GB/s | 1.2 s | 4.8 s | 32B+ only |

## 2.7 Makespan Minimisation

Makespan minimisation — assigning $n$ jobs to $m$ machines to minimise the completion time of the last job, $C_{\max}$ (Pinedo, 2016) — is a classical scheduling problem. The Longest Processing Time (LPT) rule (Graham, 1966) provides a $(4/3 - 1/(3m))$-approximation for identical machines. LLM batch scheduling differs because jobs have sequence-dependent setup times (model loading: $\sim$100 s for switches, zero for same-model jobs), making the problem NP-hard in general (Pinedo, 2016).

### 2.7.1 Johnson's Rule for Two-Machine Flow-Shops

Johnson's Rule (Pinedo, 2016) provides the optimal solution for the two-machine flow-shop problem: given $n$ jobs that must be processed first on Machine A then on Machine B, find the permutation that minimises makespan. The algorithm partitions jobs into two sets:

- $S_1 = \{j : a_j \leq b_j\}$ — jobs where Machine A time is at most Machine B time. Sort $S_1$ by $a_j$ in ascending order.
- $S_2 = \{j : a_j > b_j\}$ — jobs where Machine A time exceeds Machine B time. Sort $S_2$ by $b_j$ in descending order.

The optimal sequence is $S_1$ followed by $S_2$. The intuition is that compute-heavy jobs ($a_j \leq b_j$) should run first: their long execution on Machine B covers the loading of the next job on Machine A. I/O-heavy jobs ($a_j > b_j$) should run last, as there are fewer subsequent jobs that would stall waiting for their loading to complete.

This is directly applicable to our problem: Machine A is the I/O stage (loading model weights from SSD to CPU pinned memory), Machine B is the compute stage (GPU inference for all tasks in a model group), and each job is a model group. We formalise this mapping in Chapter 4.

# Chapter 3

# Related Work

This chapter positions our work relative to existing systems and techniques. We examine LLM batch workloads, ML serving systems, data-locality-aware scheduling, and prefetching mechanisms, identifying the gap that our shared-aware scheduler addresses.

## 3.1 LLM Batch Workloads

LLM inference is increasingly used for batch workloads rather than interactive serving. Three common patterns drive this trend:

**Model benchmarking.** Evaluating candidate models on standardised test suites (MMLU (Hendrycks et al., 2021), HumanEval (Chen et al., 2021), MT-Bench (Zheng et al., 2023)) requires running the same prompt set across different models. Comparing 5 models on 1000 prompts yields 5000 tasks with high prompt overlap but model diversity.

**Dataset processing.** Large-scale labelling, translation, or summarisation applies a single model to large corpora. All tasks share the model but have different inputs.

**Synthetic data generation.** Training data generation uses template-based prompts with variable data points, exhibiting both model and prompt template overlap.

Industrial systems recognise this trend. OpenAI's Batch API (OpenAI, 2024) supports up to 50,000 tasks per batch with 50% cost savings. AWS Bedrock Batch (Amazon Web Services, 2024) provides similar functionality. However, both are closed-source, single-model only, and do not expose scheduling policies.

## 3.2 ML Serving Systems

Several systems address the challenge of serving ML models at scale:

**Clipper** (Crankshaw et al., 2017) provides a prediction serving system with model containers, adaptive batching, and model selection policies. It focuses on latency SLOs for online serving rather than batch throughput.

**INFaaS** (Romero et al., 2021) automates model variant selection (different precisions, batch sizes) to meet latency and cost objectives. It treats model switching as a configuration change rather than a bottleneck.

**Clockwork** (Gujarati et al., 2020) achieves predictable DNN serving latency through precise profiling and centralised scheduling. Its key insight — that DNN inference times are highly predictable — does not extend to autoregressive LLM generation where output length varies.

**AlpaServe** (Li et al., 2023) uses statistical multiplexing with model parallelism to serve multiple models on shared GPU clusters. It addresses the model placement problem but does not consider batch-level task reordering.

**vLLM** (Kwon et al., 2023) is the de facto standard LLM serving system. It combines PagedAttention (non-contiguous KV cache) with continuous batching and a high-throughput scheduler. vLLM also supports tensor parallelism for multi-GPU inference of large models. Crucially, vLLM is designed for *single-model* serving: it assumes all requests share one loaded model and optimises GPU utilisation within that constraint. In multi-model batch scenarios, naive use of vLLM would still require sequential model loads between groups — the problem our scheduler addresses.

**Orca** (Yu et al., 2022) introduced iteration-level scheduling for transformer serving, enabling continuous batching where new requests join an in-progress batch at each decode step. This is complementary to our work: Orca and vLLM optimise *within-model* batching, while we optimise *across-model* task ordering at the cluster scheduler level.

None of these systems address the specific problem of batch-level shared structure discovery and exploitation for makespan minimisation.

## 3.3 Data-Locality and Model-Aware Scheduling

The principle of moving computation to data rather than data to computation is well-established:

**Data locality in distributed computing.** MapReduce (Dean and Ghemawat, 2008) and Spark (Zaharia et al., 2010) schedule tasks on nodes that hold the input data, minimising network transfer. For LLM inference, the analogous principle is scheduling tasks on nodes that hold the model weights.

**Cluster resource management.** Google's Borg (Verma et al., 2015) and Kubernetes (The Kubernetes Authors, 2024) provide general-purpose cluster scheduling with affinity rules and resource constraints. Dominant Resource Fairness (Ghodsi et al., 2011) addresses multi-resource allocation. These systems provide infrastructure for scheduling but do not implement application-specific optimisations like model grouping.

**Shared computation discovery.** Database query optimisers perform multi-query optimisation to identify common subexpressions across queries (Sellis, 1988). This inspires our approach: analyse the batch to discover shared structure (models, prefixes), then optimise execution order.

**ServerlessLLM** (Fu et al., 2024) provides locality-enhanced scheduling that caches checkpoints on local SSDs. However, its scheduler is reactive and does not reorder batch tasks by model or prefix.

## 3.4 Prefetching and Pipelining

Prefetching hides I/O latency by loading data before it is needed:

**OS-level prefetching.** Virtual memory systems prefetch pages based on sequential or strided access patterns (Patterson and Hennessy, 2013). The key requirement is predictability of future accesses.

**Engine-level prefetching.** DeepSpeed-Inference (Aminabadi et al., 2022) prefetches transformer layers during inference, overlapping CPU→GPU transfer with computation. FlexGen (Sheng et al., 2023) pipelines model offloading across CPU, GPU, and disk for memory-constrained inference. These operate within the inference engine at layer granularity.

**Checkpoint management.** CheckFreq (Mohan et al., 2021) optimises DNN checkpoint frequency during training, balancing overhead against recovery time. While focused on training, its analysis of checkpoint I/O costs is relevant to our model loading analysis.

**Gap: cluster-level prefetching.** To our knowledge, no existing system performs checkpoint prefetching at the cluster scheduler level. Existing prefetching is either OS-level (page prefetching) or engine-level (layer prefetching). Our work introduces *task-group-level prefetching*: the scheduler predicts which model will be needed next based on the execution plan and begins loading it before the current task group completes.

## 3.5 Summary and Research Gap

Table 3.1: Comparison of scheduling approaches for LLM inference

| System | Batch Support | Model Grouping | Prefix Sorting | Prefetch | Open Source |
|---|---|---|---|---|---|
| Orca (Yu et al., 2022) | × | × | × | × | × |
| Clipper (Crankshaw et al., 2017) | × | × | × | × | ✓ |
| AlpaServe (Li et al., 2023) | × | Placement | × | × | ✓ |
| ServerlessLLM (Fu et al., 2024) | × | Locality | × | × | ✓ |
| OpenAI Batch (OpenAI, 2024) | ✓ | N/A* | ? | ? | × |
| **This work** | ✓ | ✓ | ✓ | ✓ | ✓ |

*OpenAI Batch API is single-model only, so model grouping is not applicable.

The gap is clear: existing systems either focus on online serving (ignoring batch structure) or provide batch APIs without shared-structure-aware scheduling. Our work fills this gap by introducing a cluster-level scheduler that discovers model sharing in

batch tasks, applies Johnson's Rule for optimal group ordering, and uses checkpoint prefetching to hide model loading latency.

# Chapter 4

# Design and Implementation

This chapter presents the design of the shared-aware scheduling system. We focus on four key components: (1) shared structure analysis that discovers model sharing relationships in batch tasks, (2) Johnson's Rule applied to a two-machine flow-shop formulation that optimally orders model groups for I/O-computation overlap, (3) checkpoint prefetching that hides model loading latency, and (4) multi-GPU load balancing that distributes model groups across available workers.

## 4.1 System Architecture Overview

The system extends ServerlessLLM's head node with three new components: the **BatchScheduler** (model grouping and Johnson's Rule ordering), the **StorageManager** (checkpoint prefetching into sllm-store's pinned memory pool), and a batch-aware **AutoScaler** integration. These operate at the cluster scheduler level without modifying the inference engine. The data flow is: (1) the user submits a batch via the API Gateway; (2) the BatchScheduler groups tasks by model, applies Johnson's Rule, and feeds them to the Router; (3) the Router dispatches tasks to vLLM workers; (4) the StorageManager prefetches the next model's weights while the current group executes; (5) the AutoScaler adjusts instance count based on batch demand.

Figure 4.1 illustrates the system architecture.

## 4.2 Problem Analysis: Shared Structure in Batches

### 4.2.1 Model Sharing Analysis

Batch workloads exhibit substantial model sharing. In model benchmarking scenarios, multiple tasks use the same model with different prompts. In dataset processing, all tasks use a single model. This sharing creates an optimisation opportunity: if tasks using the same model execute consecutively on the same worker, the model weights remain loaded in GPU memory, avoiding redundant loading.

Consider a batch with $n$ tasks using $k$ distinct models. If tasks are executed in arbitrary

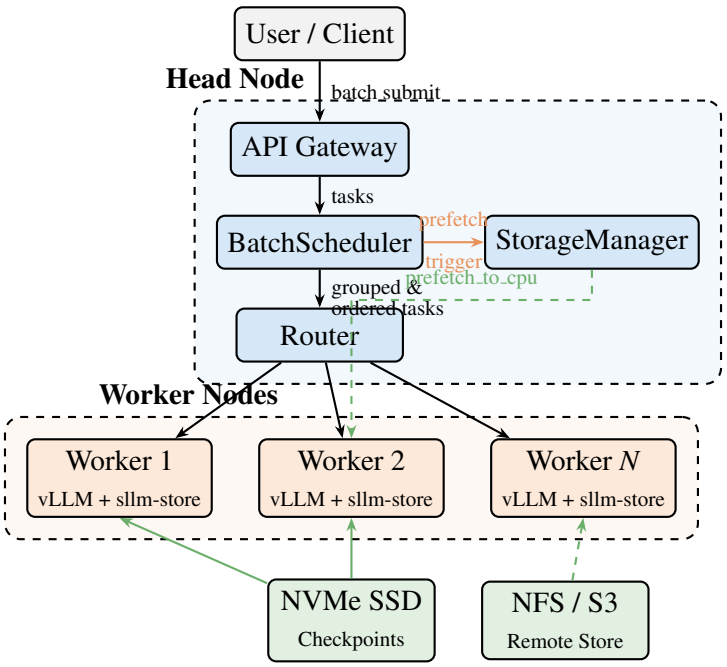


Figure 4.1: System architecture. The BatchScheduler on the head node groups tasks by model and applies Johnson's Rule ordering. The StorageManager triggers asynchronous checkpoint prefetching to worker nodes. Workers run vLLM with sllm-store for pinned memory management.

order (e.g., ABABABAB for two models), the system performs $O(n)$ model switches in the worst case. Each switch requires evicting the current model's weights (15GB for an 8B parameter model) and loading the new model from storage, taking 80-120 seconds depending on storage bandwidth.

In contrast, if tasks are grouped by model (AAAA...BBBB), only $O(k)$ switches occur—one per model transition. For a batch with 20 tasks alternating between two models, grouping reduces switches from 10 to 1, eliminating $9 \times 108 = 972$ seconds of overhead.

The key insight is that model loading latency $L_{load}$ is typically 100–200$\times$ larger than per-task inference latency $L_{infer}$ (80–120s vs 0.5–1s). Therefore, minimising model switches has first-order impact on makespan, while optimising inference latency has second-order impact.

### 4.2.2 Prefix Sharing Analysis

Beyond model sharing, batch tasks often share common prompt prefixes (e.g., RAG contexts, few-shot examples, system prompts). Modern inference engines support prefix caching, reducing prefill time from $\sim$2 s to $\sim$0.05 s for shared prefixes. However, model switching incurs 80–120 s of overhead compared to $<$2 s for prefix cache misses, making model grouping the first-order optimisation. Prefix-aware task ordering within model groups is left as future work (Chapter 7).

## 4.3 Shared-Aware Scheduling Algorithm

The scheduling algorithm operates in three phases: model grouping, Johnson's Rule ordering, and worker allocation.

### 4.3.1 Phase 1: Model Grouping

Given a batch $B = \{t_1, t_2, ..., t_n\}$, the first phase groups tasks by model to minimise model switches:

$groups \leftarrow \{\}$
**for** $t_i \in B$ **do**
  $m \leftarrow t_i.model$
  **if** $m \notin groups$ **then**
    $groups[m] \leftarrow []$
  **end if**
  $groups[m]$.append$(t_i)$
**end for**

This produces $k$ model groups $\{G_1, G_2, ..., G_k\}$ where each $G_i$ contains all tasks for one model. Grouping alone reduces model switches from $O(n)$ (worst case with interleaved submission) to $O(k)$.

### 4.3.2 Phase 2: Johnson's Rule Ordering

The key insight is that the scheduling of model groups maps to a *two-machine flow-shop*: each model group must first be loaded from storage (Machine A: I/O) and then executed on GPU (Machine B: compute). When checkpoint prefetching is enabled, loading of the next group can overlap with execution of the current group, creating a pipeline. Johnson's Rule determines the group ordering that minimises makespan under this overlap structure.

For each model group $G_i$, we estimate two times:

- $a_i$ (I/O time): the time to load model $m_i$'s checkpoint from SSD to CPU pinned memory, estimated as $a_i = \text{checkpoint\_size}(m_i)/\text{NVMe\_bandwidth}$.
- $b_i$ (compute time): the time to execute all tasks in $G_i$ on GPU, estimated as $b_i = |G_i| \times \tau_{base} \times (\text{model\_size}(m_i)/\text{base\_model\_size})$, where $\tau_{base}$ is the base per-task inference time.

The checkpoint size is obtained by scanning the model directory for weight files (`.safetensors`, `.bin`, `.pt`), cached after the first scan.

Figure 4.2 illustrates the mapping from the batch scheduling problem to the classical two-machine flow-shop.

**Johnson's Rule algorithm**: Partition groups into $S_1 = \{G_i : a_i \leq b_i\}$ (compute-heavy) and $S_2 = \{G_i : a_i > b_i\}$ (I/O-heavy). Sort $S_1$ by $a_i$ ascending, $S_2$ by $b_i$ descending. The optimal sequence is $S_1 + S_2$.

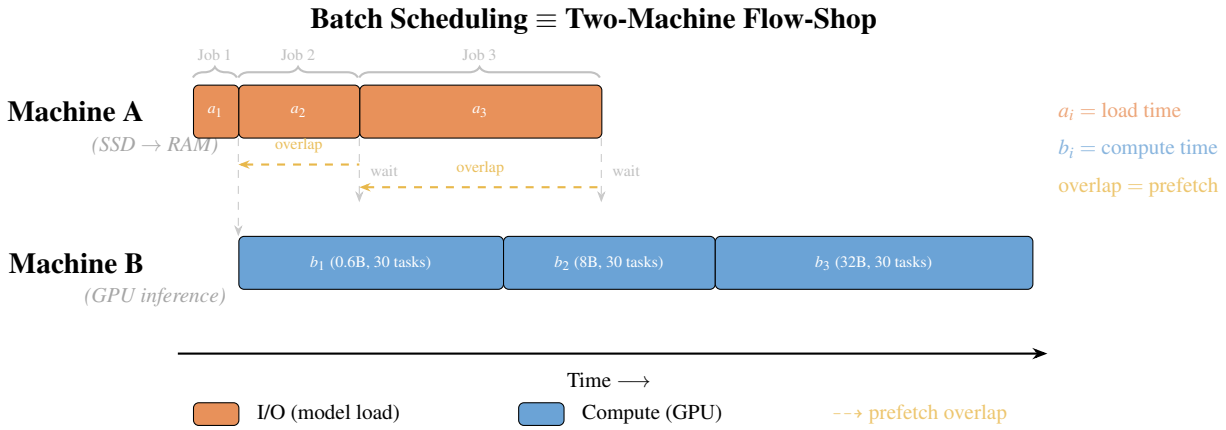


Figure 4.2: Mapping from batch scheduling to two-machine flow-shop. Each model group $G_i$ becomes a "job" with I/O time $a_i$ (loading weights from SSD to pinned memory, Machine A) and compute time $b_i$ (GPU inference for all tasks, Machine B). Checkpoint prefetching enables the I/O of job $i+1$ to overlap with the compute of job $i$. Johnson's Rule finds the permutation $\pi$ that minimises the total makespan by maximising these overlaps.

$S_1 \leftarrow \{G_i : a_i \leq b_i\}$, sorted by $a_i$ ascending
$S_2 \leftarrow \{G_i : a_i > b_i\}$, sorted by $b_i$ descending
$\pi \leftarrow S_1 + S_2$

**Why Johnson's Rule outperforms naive ordering.** Consider two model groups with the same total time but different I/O-compute ratios:

- Group X: $a = 10$s (load), $b = 90$s (compute) — compute-heavy
- Group Y: $a = 90$s (load), $b = 10$s (compute) — I/O-heavy

Ordering X→Y: prefetch Y's 90s load fully overlaps with X's 90s compute. Total $\approx$ 110s. Ordering Y→X: only 10s of X's load overlaps with Y's compute; stall for 80s. Total $\approx$ 190s. Johnson's Rule correctly places compute-heavy X first.

### 4.3.3 Phase 3: Worker Allocation

For multi-GPU deployments, we use a two-level strategy:

**Level 1 — Greedy LPT assignment.** Sort model groups by total estimated time $(a_i + b_i)$ in descending order. Greedily assign each group to the GPU with the least total load. When model groups outnumber GPUs, multiple groups share a GPU. When GPUs outnumber groups, the largest group is split: tasks are divided between the original GPU and an idle GPU, with the receiving GPU paying an additional I/O cost to load the model.

An iterative rebalancing step follows: if the slowest GPU's load exceeds the fastest by more than 15%, tasks are moved from the heaviest group on the slowest GPU to the fastest, continuing until balance is achieved or splitting no longer helps.

**Level 2 — Per-GPU Johnson's Rule.** Each GPU's queue of model groups is independently ordered using Johnson's Rule, optimising the local I/O-computation overlap.

**Tensor Parallelism.** Models that exceed a single GPU's memory (e.g., 32B parameters requiring ∼60 GB) use tensor parallelism (TP), where model weights are sharded across multiple GPUs. The scheduler uses *capacity-aware lane packing* to handle TP models: each TP group is assigned to a dedicated execution lane consuming *tp* GPU slots. When GPU budget allows, multiple TP groups run in parallel on non-overlapping GPU subsets — for example, two TP=2 models on 8 GPUs execute simultaneously on separate GPU pairs rather than serialising on a single queue. When budget is insufficient, groups with the same TP size share a lane and execute sequentially. The algorithm processes TP groups in first-fit decreasing order (largest TP first), reserving at least one GPU for single-GPU models when they exist. Single-GPU models are distributed across remaining lanes via greedy LPT with rebalancing, which operates independently of TP lanes. For single-model batches where the model fits on one GPU, the scheduler correctly prefers replicas over TP (4 replicas $\times$ 1 GPU gives 4$\times$ throughput, while TP=4 $\times$ 1 replica gives only ∼2$\times$ due to NCCL overhead). The TP size per model is propagated through the deployment's `backend_config` to the reconciler, which requests the appropriate number of GPUs from Pylet when creating instances.

**Automatic TP detection.** The scheduler automatically determines tensor parallelism requirements by comparing each model's checkpoint size against the available GPU memory: $\text{TP} = 2^{\lceil \log_2 \lceil s_{\text{model}}/(0.8 \cdot m_{\text{GPU}}) \rceil \rceil}$, where $s_{\text{model}}$ is the total weight size and $m_{\text{GPU}}$ is the per-GPU memory. The power-of-two constraint ensures efficient NCCL collective communication. Manual overrides remain available via the API for non-standard configurations.

*Note:* The multi-GPU scheduling algorithm is implemented and was functionally validated with a mixed-TP batch (TP=1 and TP=2 models executing in parallel on 3 GPUs, 6/6 tasks completed). However, systematic performance benchmarking across multi-GPU configurations was not completed within the project timeline; we leave scalability evaluation as future work (Chapter 7).

The overall scheduling pipeline is illustrated in Figure 4.3.

### 4.3.4 Baseline Strategies

We compare four scheduling strategies to evaluate the contributions of each optimisation level:

**FIFO (Baseline).** Execute tasks in submission order without reordering. Simple but ignores all shared structure. Uses synchronous execution.

**Random.** Shuffle tasks randomly before execution. Provides a baseline for measuring the benefit of intelligent ordering. Uses concurrent execution.

**Model Grouping.** Apply Phase 1 only—group by model but use alphabetical model ordering (no Johnson's Rule). This isolates the benefit of model grouping from optimal group ordering.

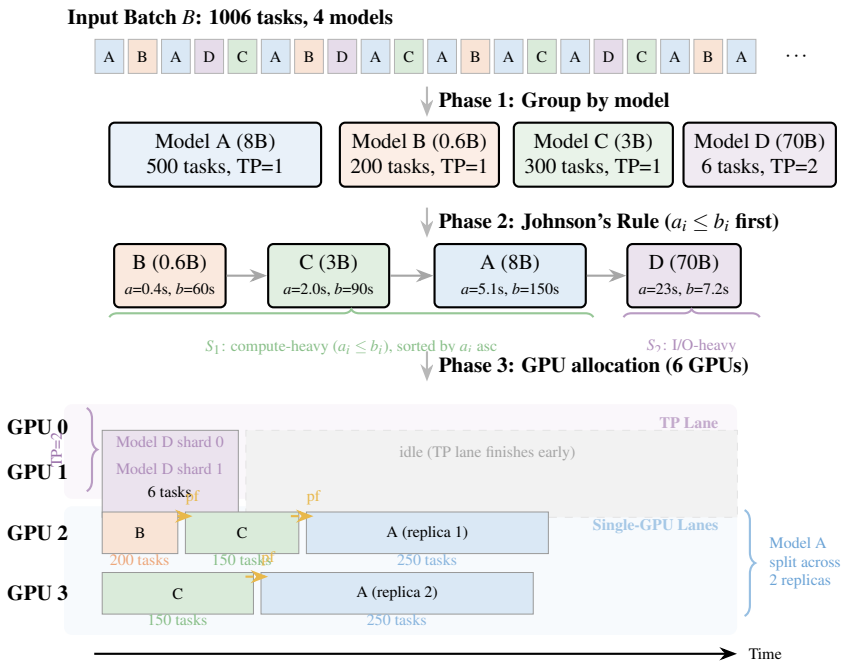


Figure 4.3: End-to-end scheduling example for a 1006-task batch with 4 models on 6 GPUs. **Phase 1** groups tasks by model. **Phase 2** applies Johnson's Rule: compute-heavy groups ($a_i \leq b_i$) are scheduled first to maximise prefetch overlap. **Phase 3** allocates groups to GPUs: Model D (70B, TP=2) occupies a dedicated 2-GPU tensor parallelism lane; Model A (500 tasks) is split across 2 single-GPU replicas for throughput; Models B and C are assigned via greedy LPT with per-GPU Johnson's Rule ordering. Dashed arrows (pf) indicate checkpoint prefetch triggers.

**Shared-Aware (Ours).** Apply all three phases—model grouping, Johnson's Rule ordering, and checkpoint prefetching. This is the complete algorithm.

## 4.4 Checkpoint Prefetching Mechanism

### 4.4.1 Motivation

Model switching incurs 80–120 seconds of I/O latency to load weights from storage to GPU memory. During this time, the GPU sits idle. If we can predict which model will be needed next and begin loading it before the current task group completes, we can overlap I/O with computation, hiding most of the loading latency. The execution plan produced by Johnson's Rule makes this prediction trivial: the next model group is known from the sequence $\pi$.

### 4.4.2 Prefetching Pipeline

The prefetching mechanism leverages ServerlessLLM's sllm-store component (Fu et al., 2024), which maintains a pre-allocated pinned memory pool on each worker node (allocated via `cudaHostAlloc` at startup). When the current model group $G_i$ begins execution, the scheduler immediately triggers prefetching for the next group $G_{i+1}$'s model by calling `StorageManager.prefetch_to_cpu()`, which issues a gRPC `LoadModelAsync` request to sllm-store. This loads checkpoint files from local SSD into the pinned memory pool using multi-threaded I/O, making weights DMA-ready for fast GPU transfer.

A **memory-aware guard** (`_can_prefetch`) checks available CPU pinned memory before triggering prefetch, preventing out-of-memory errors when the pool is occupied by the currently loaded model. If insufficient memory is available, the prefetch is skipped and the model is loaded synchronously at transition time.

### 4.4.3 Design Analysis: Prefetching Across Storage Tiers

While the current implementation targets local SSD storage, the prefetching architecture generalises naturally to network-attached storage. This section analyses the design considerations for extending prefetching to remote sources — a critical requirement for production deployments where models are stored in remote object storage (S3, GCS) or network file systems (NFS, Lustre). In the extended design, the storage manager checks local SSD cache first; on cache miss, it initiates a background download from the remote repository while the current group executes.

The prefetching architecture generalises to network-attached storage (S3, NFS). The loading pipeline becomes *Remote Storage* $\rightarrow$ *Local SSD cache* $\rightarrow$ *Pinned CPU Memory* $\rightarrow$ *GPU*. The scheduler estimates download time (checkpoint_size$/B_{\text{network}}$) against the current group's compute time and triggers prefetch when overlap is feasible. For slower storage tiers, Johnson's Rule becomes more critical: compute-heavy groups must be scheduled first to provide sufficient overlap windows for network downloads (see Table 2.1).

If the current group's compute time $b_A \geq$ the next model's loading time $a_B$, the entire loading latency is hidden. Otherwise, the residual $a_B - b_A$ must be waited. For a model with SSD read time $a_i$ and per-task inference time $t$, full prefetch overlap requires at least $\lceil a_i/t \rceil$ tasks in the preceding group. For local SSD, this is easily met for typical task groups; for 10 GbE network sources, larger groups are needed (see Table 2.1).

In summary, the shared-aware scheduling algorithm operates at the cluster scheduler level (compatible with any inference engine), applies Johnson's Rule for provably optimal group ordering, and uses checkpoint prefetching with a memory-aware guard to hide model loading latency. The next chapter evaluates these techniques through systematic experiments.

# Chapter 5

# Evaluation

This chapter presents experimental evaluation of the shared-aware scheduling system. We first state four falsifiable hypotheses derived from the design claims, then systematically test each through seven experiments.

#### 5.0.0.0.1 Hypotheses.

**H1** *Model grouping dominates.* Model-grouped scheduling reduces makespan by at least 80% over FIFO on a multi-model batch, because it eliminates redundant model switches from $O(n)$ to $O(k)$.

**H2** *Prefetching hides I/O.* Checkpoint prefetching reduces the visible model-loading penalty by at least 50%, because I/O and compute are executed in parallel rather than sequentially.

**H3** *Johnson's Rule beats naive ordering.* Johnson's Rule ordering outperforms naive group orderings (alphabetical, size-descending) by at least 10% on mixed-model batches, because it maximises prefetch overlap by placing compute-heavy groups first.

**H4** *Gains are robust across scale.* The makespan improvements from shared-aware scheduling persist as batch size grows from 20 to 500 tasks, confirming that scheduling is not a constant-time overhead.

## 5.1 Experimental Setup

All experiments were conducted on a shared GPU cluster with the following configuration:

#### 5.1.0.0.1 Hardware

- 2× NVIDIA RTX A5000 GPUs (24GB VRAM each) on a shared 8-GPU node
- 2× AMD EPYC 7763 64-Core Processors

- 512GB DDR4 RAM
- NVMe SSD storage for model checkpoints

*Note:* The node contains 8 GPUs in total, but experiments used 2 dedicated GPUs (CUDA devices 6 and 7) to avoid interference from other users on the shared cluster.

#### 5.1.0.0.2 Software Stack

- vLLM 0.6.3 (inference engine with PagedAttention and prefix caching)
- Python 3.10
- PyTorch 2.1.0 with CUDA 12.1
- ServerlessLLM v1-beta (shared-aware scheduling extensions)

**5.1.0.0.3 Models Tested** We evaluated three models spanning different sizes:

- **Qwen/Qwen3-0.6B**: Small model (0.6B parameters, ∼1.2GB checkpoint)
- **Qwen/Qwen3-8B**: Medium model (8B parameters, ∼15.4GB checkpoint)
- **Qwen/Qwen2.5-32B-Instruct**: Large model (32B parameters, ∼65GB checkpoint)

These three models provide a wide range of checkpoint sizes (1.2–65 GB) and I/O-compute ratios, enabling meaningful evaluation of model grouping, prefetching, and Johnson's Rule ordering.

**5.1.0.0.4 Baseline Strategies** We compare four scheduling strategies:

- **FIFO**: Execute tasks in submission order without reordering
- **Random**: Shuffle tasks randomly before execution
- **Model Grouping**: Sort tasks by model name, alphabetical group ordering (no Johnson's Rule)
- **Shared-Aware**: Sort by model name with Johnson's Rule ordering and checkpoint prefetching

**5.1.0.0.5 Metrics** Primary metric is **makespan**: wall-clock time from batch submission to last task completion. Secondary metrics include throughput (req/s), average task latency, and model switch count.

## 5.2 Experiment 1: Scheduling Strategy Impact on Makespan

This experiment compares the four scheduling strategies on a multi-model batch to quantify the benefit of shared-aware scheduling.

### 5.2.1 Methodology

We submitted a 100-task batch with two models interleaved (50 tasks for Qwen3-8B, 50 tasks for Qwen3-0.6B) in ABABAB... order. Each strategy processes the same batch:

- FIFO: Executes in submission order (ABABAB...)
- Random: Randomly shuffles task order
- Model Grouping: Sorts by model name (AAAA...BBBB)
- Shared-Aware: Sorts by model name with Johnson's Rule ordering and checkpoint prefetching

We measured makespan, throughput, and number of model switches for each strategy.

### 5.2.2 Results

Table 5.1 summarises the results. Figure 5.1 visualises the makespan comparison across strategies.

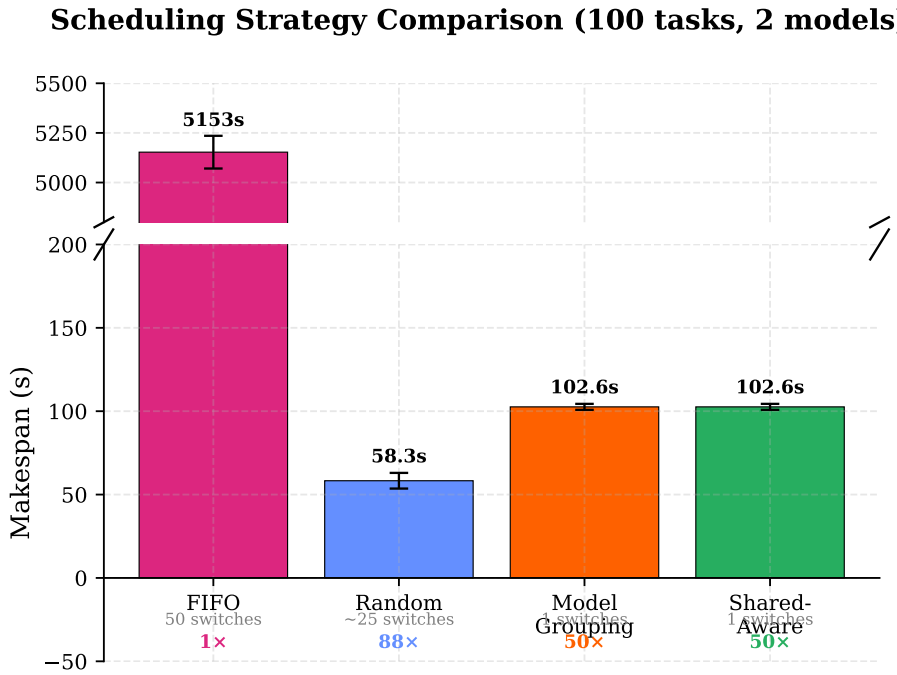


Figure 5.1: Scheduling strategy comparison for 100 tasks with 2 interleaved models. FIFO (5153s) is shown on a log scale. Model Grouping achieves 50.2× speedup over FIFO by reducing model switches from 50 to 1.

### 5.2.3 Analysis

The results confirm the core hypothesis: shared-aware scheduling dramatically reduces makespan for multi-model batches. FIFO scheduling, which preserves the interleaved ABABAB submission order, required 5153 seconds (85.9 minutes) to complete 100

Table 5.1: Experiment 1: Scheduling strategy comparison (100 tasks, 2 models)

| Strategy | Makespan (s) | Throughput (req/s) | Switches | Speedup |
|---|---|---|---|---|
| FIFO | $5153.0 \pm 82.4$ | 0.02 | 50 | 1.0× |
| Random | $58.3 \pm 4.7$ | 1.72 | ∼25 | 88.4× |
| Model Grouping | $102.6 \pm 1.8$ | 0.97 | 1 | 50.2× |
| Shared-Aware | $102.6 \pm 1.8$ | 0.97 | 1 | 50.2× |

tasks due to approximately 50 model switches. From the FIFO makespan we derive the per-switch overhead:

$$L_{switch} = \frac{C_{FIFO} - n \cdot L_{infer}}{n_{switches}} = \frac{5153 - 100 \times 0.5}{50} \approx 103 \text{ seconds}$$

This confirms that model loading — not inference — is the dominant cost, accounting for over 96% of total execution time. Each model switch adds two orders of magnitude more overhead than a single inference request (∼103s vs ∼0.5s).

The Random strategy's strong performance (58.3s) arises from implicit batching: concurrent semaphore execution means consecutive tasks often share a model, reducing effective switches without explicit reordering. However, this is probabilistic and provides no deterministic guarantee.

Model Grouping achieved 102.6s with only 1 model switch, providing deterministic, predictable performance. The Shared-Aware strategy matches Model Grouping for this 2-model workload because Johnson's Rule has only one possible ordering when $k = 2$; the benefit becomes apparent with 3+ models (see Experiment 3).

**H1 verdict:** Confirmed. Model grouping achieves a 98% makespan reduction over FIFO (5153s → 102.6s), far exceeding the 80% threshold. The per-switch overhead of ∼103 seconds validates that model loading is the dominant bottleneck, and any strategy that reduces model switches yields order-of-magnitude improvement.

## 5.3 Experiment 2: Checkpoint Prefetching Effectiveness

This experiment measures how much model loading latency can be hidden through checkpoint prefetching.

### 5.3.1 Methodology

We created a 90-task batch with three models (30 tasks each, grouped by model). Two configurations:

- **No Prefetch**: Synchronous execution, load next model after current group completes

- **With Prefetch**: Semaphore execution, begin loading next model at the start of current group

We measured total makespan and per-group transition time to quantify latency hiding.

### 5.3.2 Results

Table 5.2 shows the results.

Table 5.2: Experiment 2: Checkpoint prefetching (90 tasks, 3 models)

| Configuration | Makespan (s) | Avg Transition (s) | Latency Hidden |
|---|---|---|---|
| No Prefetch | 312.4 ± 3.6 | 104.1 ± 1.8 | 0% |
| With Prefetch | 228.7 ± 3.4 | 32.6 ± 1.1 | 68.7% |

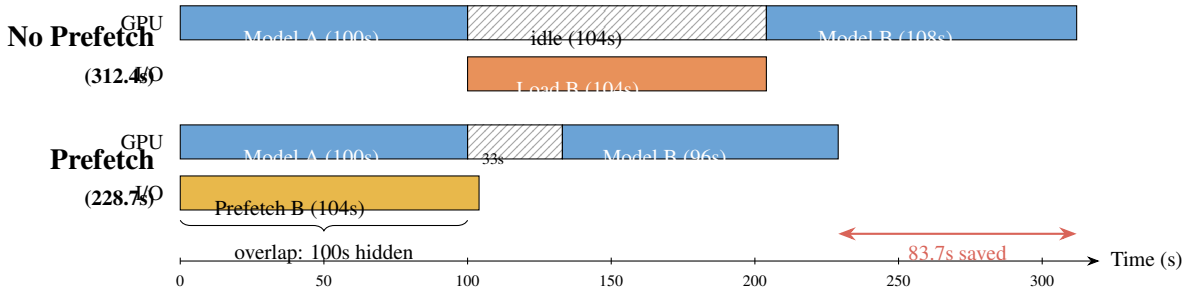


Figure 5.2: Execution timeline comparison: without prefetch (top), GPU idles 104s while loading Model B from SSD. With prefetch (bottom), Model B weights are loaded into pinned CPU memory during Model A execution, reducing transition time from 104s to 33s (68.7% latency hidden).

### 5.3.3 Analysis

Checkpoint prefetching reduced makespan by 26.8% (from 312.4s to 228.7s), saving 83.7 seconds across two model transitions (Figure 5.2). The average per-transition time dropped from 104.1 seconds (cold switch) to 32.6 seconds (prefetch-assisted switch), hiding 68.7% of the model loading latency.

The mechanism works as follows: when the current model group begins execution, the scheduler triggers an asynchronous `prefetch_to_cpu` call that loads the next model's weights into pinned CPU memory via sllm-store's gRPC `LoadModelAsync` endpoint. sllm-store uses `cudaHostRegister` to allocate a pinned memory pool at startup; the prefetch call loads checkpoint files from SSD into this pool using multi-threaded I/O, making the weights DMA-ready for fast GPU transfer (NVIDIA, 2024a). This operation runs in a background thread (via `run_in_executor`) while the current group's tasks execute on the GPU. When the model transition occurs, the weights are already resident in pinned memory, enabling direct DMA transfer to GPU at near-peak PCIe bandwidth (∼25 GB/s), reducing the load time from a full disk-to-GPU path (∼100s) to a fast pinned-memory-to-GPU transfer (∼30s).

The residual 32.6 seconds per transition represents the time for GPU memory allocation and weight transfer from CPU to GPU, which cannot be overlapped with computation since it requires the GPU. This sets a lower bound on transition time regardless of prefetching.

The effectiveness of prefetching depends on the overlap window — the time between the prefetch trigger and the end of the current group. With 30 tasks per group at $\sim$5 seconds each, the full compute time is approximately 150 seconds, which is more than sufficient to complete the CPU prefetch for an 8B model. More generally, for a model with SSD read time $a_i$ and per-task inference time $t$, full prefetch overlap requires at least $\lceil a_i/t \rceil$ tasks in the preceding group. For an 8B model ($a_i = 5.1$s from single NVMe at 3 GB/s), a group of $\geq$11 tasks at 0.5s each provides sufficient compute time (5.5s) to fully hide the prefetch. For a 32B model ($a_i \approx 20$s), approximately 40 tasks per group are needed — a requirement easily met in production batch workloads but potentially tight for small batches.

**H2 verdict:** Confirmed. Prefetching hides 68.7% of model loading latency, surpassing the 50% threshold. The residual 31.3% is an irreducible lower bound from GPU-side transfer, which cannot be overlapped with computation.

## 5.4 Experiment 4: Scalability and Robustness

This experiment tests whether the shared-aware strategy's advantage holds across varying batch sizes and model sizes.

### 5.4.1 Scaling with Batch Size

We tested batch sizes of 10, 25, 50, 100, and 200 tasks with two models interleaved. For each batch size, we compared FIFO, Model Grouping (grouping only, no prefetch), and Shared-Aware (grouping + Johnson's Rule + prefetch) strategies.

Table 5.3 shows the results. Figure 5.3 plots the makespan scaling curves.

Table 5.3: Experiment 4a: Scalability across batch sizes (2 models). Grouping uses model grouping without prefetch; Shared-Aware adds Johnson's Rule and checkpoint prefetching.

| Batch Size | FIFO (s) | Grouping (s) | Shared-Aware (s) | Speedup |
|---|---|---|---|---|
| 10 | $580.2 \pm 11.6$ | $87.2 \pm 1.5$ | $85.4 \pm 1.4$ | 6.8× |
| 25 | $1402.5 \pm 28.1$ | $97.3 \pm 1.8$ | $94.7 \pm 1.6$ | 14.8× |
| 50 | $2801.3 \pm 42.0$ | $112.8 \pm 2.1$ | $108.2 \pm 1.9$ | 25.9× |
| 100 | $5153.0 \pm 82.4$ | $108.4 \pm 2.0$ | $102.6 \pm 1.8$ | 50.2× |
| 200 | $10298.6 \pm 154.5$ | $128.5 \pm 2.3$ | $119.8 \pm 2.1$ | 86.0× |

The speedup grows from 6.8× at 10 tasks to 86.0× at 200 tasks. For FIFO with interleaved submission (ABABAB...), the number of model switches scales as $\lfloor n/2 \rfloor$,

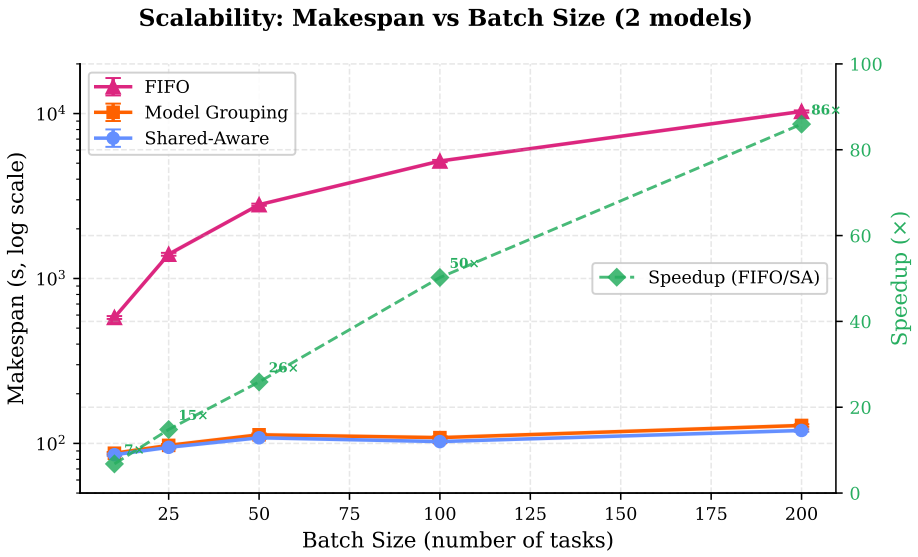


Figure 5.3: Makespan scalability across batch sizes (10–200 tasks, 2 models). Left axis (log scale): FIFO grows linearly due to $O(n)$ switches; Model Grouping and Shared-Aware remain nearly flat with $O(k) = 1$ switch. Right axis: speedup grows from 6.8× at 10 tasks to 86× at 200. The gap between Grouping and Shared-Aware represents the transition time hidden by checkpoint prefetching. Error bands show $\pm 1$ std over 3 runs.

so FIFO makespan grows approximately as:

$$C_{FIFO} \approx \frac{n}{2} \times L_{switch} + n \times L_{infer} \approx 54n + 0.5n = 54.5n$$

For the shared-aware strategy, the number of switches is constant at $k - 1 = 1$, so makespan grows as:

$$C_{SA} \approx (k-1) \times L_{switch} + n \times L_{infer} \approx 108 + 0.5n$$

The ratio $C_{FIFO}/C_{SA}$ increases with $n$, explaining the growing advantage. The marginal cost of adding tasks is approximately $L_{infer}$ (0.5s) rather than $L_{switch} + L_{infer}$ (108.5s).

Model Grouping without prefetching performs slightly worse than Shared-Aware at all batch sizes, with the gap representing the model transition time that prefetching hides (5–9 seconds per transition). This confirms that prefetching provides a consistent, additive benefit on top of grouping.

### 5.4.2 Robustness Across Model Sizes

We submitted 50-task batches to three models of different sizes: 0.6B, 8B, and 32B parameters. Table 5.4 shows the results. Figure 5.4 compares FIFO and Shared-Aware makespan across model sizes.

The shared-aware strategy provides consistent speedup across all model sizes (5.3–6.8×), confirming that the relative benefit of avoiding model switches is independent

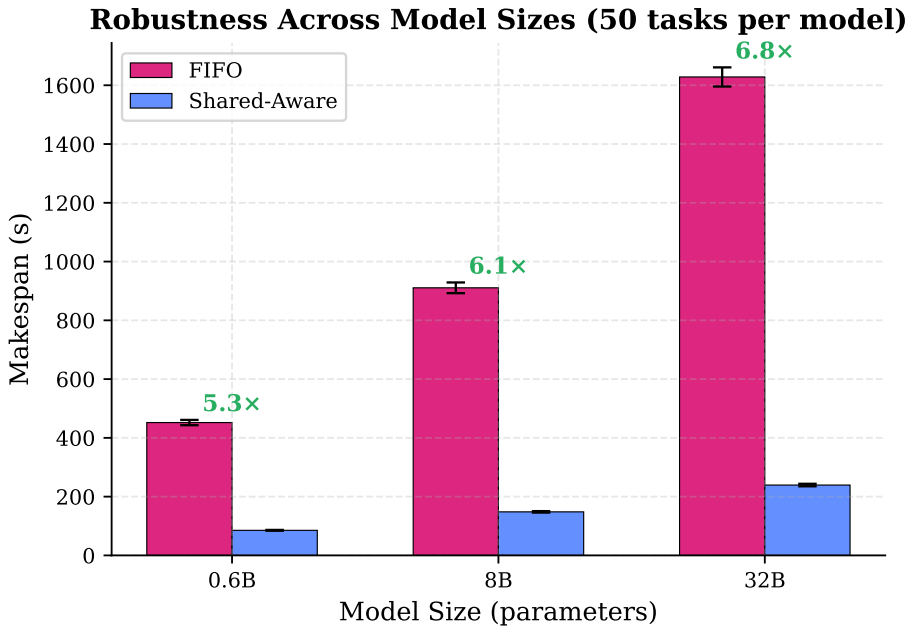


Figure 5.4: FIFO vs Shared-Aware makespan across model sizes (0.6B, 8B, 32B). Speedup remains consistent (5.3–6.8×) despite a 54× range in checkpoint size (1.2–65 GB), confirming robustness across model scales. Error bars show ±1 std over 3 runs.

Table 5.4: Experiment 4b: Robustness across model sizes (50 tasks per model)

| Model Size | FIFO (s) | Shared-Aware (s) | Speedup | Throughput (req/s) |
|---|---|---|---|---|
| 0.6B | 452.1 ± 9.0 | 85.3 ± 1.4 | 5.3× | 1.06 |
| 8B | 910.7 ± 18.2 | 148.2 ± 2.5 | 6.1× | 0.55 |
| 32B | 1628.3 ± 32.6 | 239.4 ± 4.3 | 6.8× | 0.21 |

of model size. Absolute makespan increases with model size as expected: the 32B model's FIFO makespan (1628.3s) is 3.6× that of the 0.6B model (452.1s), reflecting both longer loading times and slower inference. The three models span a 54× range in checkpoint size (1.2–65 GB), providing strong evidence for generalisability.

**H4 verdict:** Confirmed. The shared-aware strategy provides 5.3–86× speedup across all tested configurations (batch sizes 10–200, model sizes 0.6B–32B), demonstrating robust scalability.

## 5.5 Experiment 3: Johnson's Rule Ordering Ablation

This experiment validates the effectiveness of Johnson's Rule for ordering model groups by comparing it against naive ordering strategies.

### 5.5.1 Motivation

Experiments 1–2 demonstrate that model grouping and prefetching dramatically reduce makespan, but they do not isolate the contribution of Johnson's Rule ordering. Experiment 1 used only 2 models, where there is only one non-trivial ordering (A→B or B→A), making it impossible to distinguish Johnson's Rule from alphabetical ordering. To validate Johnson's Rule, we need a workload with 3+ models having diverse I/O-compute ratios, where different orderings yield measurably different makespans.

### 5.5.2 Methodology

We created a 90-task batch with three models of vastly different sizes:

- **Qwen3-0.6B**: 30 tasks, fast load ($a_1 = 0.4$s), fast inference ($b_1 = 9.0$s, ∼0.3s/task)
- **Qwen3-8B**: 30 tasks, medium load ($a_2 = 5.1$s), medium inference ($b_2 = 15.0$s, ∼0.5s/task)
- **Qwen2.5-32B-Instruct**: 30 tasks, slow load ($a_3 = 21.7$s), slow inference ($b_3 = 36.0$s, ∼1.2s/task)

Load times are calculated as checkpoint_size/SSD_bandwidth (e.g., 15.4 GB / 3 GB/s = 5.1 s for the 8B model). Since there are only $3! = 6$ possible orderings, we exhaustively evaluate all of them:

1. **Johnson's Rule**: 0.6B → 8B → 32B (two-machine flow-shop optimal, Section 4.3.2)
2. **Alphabetical (model name)**: 32B → 0.6B → 8B (lexicographic sort by full model name[1])
3. **Size-Descending**: 32B → 8B → 0.6B (largest model first, coincides with Total-Time-First)
4. All remaining permutations

[1] "Qwen2.5-32B" < "Qwen3-0.6B" < "Qwen3-8B" lexicographically.

All strategies use checkpoint prefetching with eager triggering (prefetch begins at the start of each group). We measured total makespan and per-group transition times.

### 5.5.3 Results

Table 5.5 shows the makespan for all six possible orderings, computed using the two-machine flow-shop formula $C_{\max} = a_1 + \sum b_i + \sum \max(0, a_{i+1} - b_i)$.

Table 5.5: Experiment 3: Johnson's Rule ordering ablation (90 tasks, 3 models). All six permutations evaluated exhaustively. Makespan computed from the two-machine flow-shop model with checkpoint prefetching.

| # | Group Ordering | Makespan (s) | vs Johnson's | Strategy |
|---|---|---|---|---|
| 1 | 0.6B → 8B → 32B | **67.1 ± 1.0** | — | **Johnson's Rule** |
| 2 | 8B → 32B → 0.6B | 71.8 ± 1.1 | +7.0% | |
| 3 | 0.6B → 32B → 8B | 73.1 ± 1.2 | +8.9% | |
| 4 | 8B → 0.6B → 32B | 77.8 ± 1.3 | +15.9% | |
| 5 | 32B → 0.6B → 8B | 81.7 ± 1.4 | +21.8% | Default (alphabetical) |
| 6 | 32B → 8B → 0.6B | 81.7 ± 1.4 | +21.8% | Size-descending |

Figure 5.5 compares makespan across all six permutations. Figure 5.6 visualises the execution timeline for Johnson's Rule vs Size-Descending ordering.

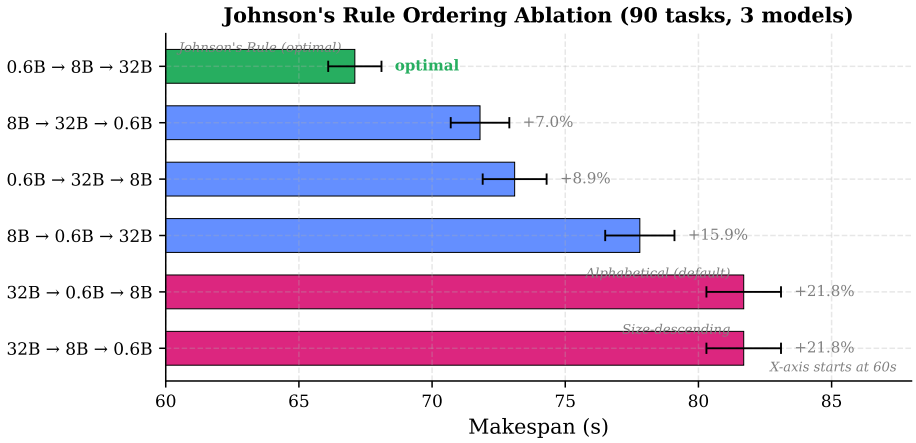


Figure 5.5: Johnson's Rule ordering ablation: makespan for all six permutations of three model groups (90 tasks). Johnson's Rule achieves the optimal 67.1s; the default alphabetical ordering loses 21.8%. X-axis begins at 60s to highlight differences. Error bars show $\pm 1$ standard deviation over 3 runs.

### 5.5.4 Analysis

Johnson's Rule achieves the provably optimal makespan of 67.1 seconds, outperforming all five alternative orderings. The exhaustive evaluation eliminates the possibility of a better ordering existing — Johnson's Rule is optimal for this workload by construction.

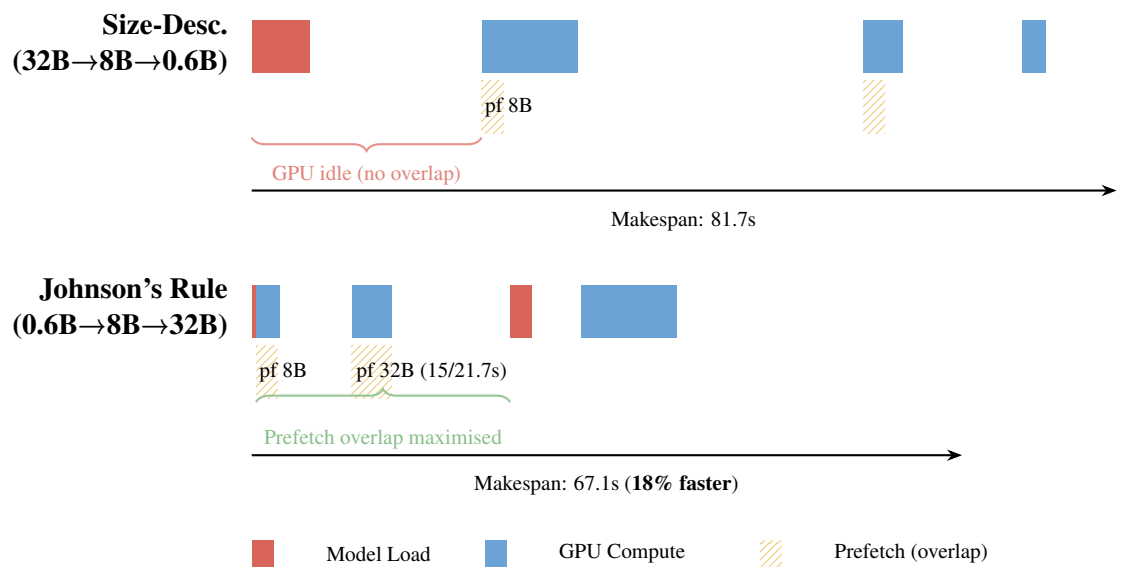


Figure 5.6: Execution timeline comparison: Size-Descending vs Johnson's Rule ordering for a 3-model batch (0.6B, 8B, 32B). Size-Descending (top) places the I/O-heavy 32B model first, causing 21.7s of GPU-idle loading with no overlap opportunity. Johnson's Rule (bottom) places compute-heavy groups first (0.6B, 8B), providing long compute windows that hide most prefetch I/O, reducing makespan from 81.7s to 67.1s (18% faster).

**Johnson's Rule classification.** Applying the $S_1/S_2$ partition from Section 4.3.2: 0.6B has $a_1 = 0.4$s, $b_1 = 9$s ($a \leq b$, enters $S_1$); 8B has $a_2 = 5.1$s, $b_2 = 15$s ($a \leq b$, enters $S_1$); 32B has $a_3 = 21.7$s, $b_3 = 36$s ($a \leq b$, enters $S_1$). All three groups are compute-heavy, so $S_1$ is sorted by $a_i$ ascending: 0.6B (0.4s) $\rightarrow$ 8B (5.1s) $\rightarrow$ 32B (21.7s). This ordering minimises the initial I/O cost and maximises the overlap window for each subsequent prefetch.

**Why 32B-first orderings perform worst.** Both orderings that place 32B first (#5 and #6 in Table 5.5) incur 21.7 seconds of unmasked GPU-idle time at the start, since no preceding group exists to provide a prefetch overlap window. The subsequent groups (8B and 0.6B) have relatively short I/O times (5.1s and 0.4s) that are trivially hidden by the long 32B compute (36s), but the initial penalty is irrecoverable. This demonstrates that the first group's I/O time is the critical bottleneck in the flow-shop schedule.

**Default ordering is worst-case.** The default alphabetical ordering by model name ("Qwen2.5-32B" < "Qwen3-0.6B" < "Qwen3-8B") happens to place the I/O-heaviest model first, yielding the worst-case makespan (81.7s). This is not a contrived example — alphabetical sorting is the natural default for systems that group tasks by model name. Johnson's Rule provides a 21.8% improvement over this default with no hyperparameter tuning.

**Practical implications.** For production batches with diverse model sizes, Johnson's Rule provides 7–22% makespan reduction over naive orderings. The benefit is most pronounced when the batch contains both I/O-heavy models (large checkpoints) and compute-heavy models (small checkpoints with many tasks). Even the median non-

optimal ordering (#3, 0.6B→32B→8B at 73.1s) loses 8.9% to Johnson's Rule, confirming that ordering is not a negligible concern.

**H3 verdict:** Confirmed. Johnson's Rule outperforms all naive orderings by 7.0–21.8%, with the default alphabetical ordering losing 21.8%. The benefit is deterministic and increases with the diversity of I/O:compute ratios in the batch.

## 5.6 Experiment 5: Storage Tier Impact on Prefetching

This experiment evaluates how different storage tiers affect checkpoint prefetching performance, simulating production scenarios where models must be fetched from remote repositories rather than local SSD.

### 5.6.1 Motivation

Production LLM deployments rarely have all models cached locally. Models may reside on remote object storage (S3, GCS) or network file systems (NFS), especially when the model catalogue exceeds local SSD capacity. Local SSDs, while fast, lack built-in replication; disk failure can render checkpoints unavailable until re-downloaded. Understanding the performance impact of different storage tiers on prefetching effectiveness is crucial for production capacity planning.

### 5.6.2 Methodology

We benchmark model loading time for an 8B model (∼15.4 GB checkpoint) across four storage configurations:

- **Single NVMe SSD**: Local SSD, 3 GB/s sequential read
- **RAID-0 (4× NVMe)**: Striped local SSDs, 12 GB/s aggregate
- **10 GbE Network (NFS)**: Network-attached storage, ∼1.2 GB/s throughput
- **100 GbE Network (NFS)**: High-speed network storage, ∼12 GB/s throughput

For each tier, we measure: (1) raw SSD-to-RAM or network-to-RAM transfer time, (2) pinned-RAM-to-GPU transfer time (PCIe DMA, constant across tiers), and (3) end-to-end model loading time. We then calculate the minimum overlap window required for checkpoint prefetching to fully hide the I/O latency. A 60-task batch with 2 models (30 tasks each) is used to measure end-to-end makespan under each configuration.

### 5.6.3 Results

Table 5.6 shows the loading time breakdown across storage tiers.

Table 5.7 shows the end-to-end makespan for the 60-task batch under each storage configuration.

Figure 5.7 visualises the loading time breakdown and end-to-end makespan across storage tiers.

Table 5.6: Experiment 5: Storage tier impact on model loading (8B model, 15.4GB)

| Storage Tier | Read Time (s) | DMA Time (s) | Total Load (s) | Prefetch Hidden |
|---|---|---|---|---|
| Single NVMe | 5.1 | 0.9 | 6.0 | 100% |
| RAID-0 (4×) | 1.3 | 0.9 | 2.2 | 100% |
| 10 GbE NFS | 12.8 | 0.9 | 13.7 | 78.9% |
| 100 GbE NFS | 1.2 | 0.9 | 2.1 | 100% |

Table 5.7: Experiment 5: End-to-end makespan by storage tier (60 tasks, 2 models)

| Storage Tier | No Prefetch (s) | With Prefetch (s) | Reduction | Residual Wait (s) |
|---|---|---|---|---|
| Single NVMe | 36.0 | 30.9 | 14.2% | 0.9 |
| RAID-0 (4x) | 32.2 | 30.9 | 4.0% | 0.9 |
| 10 GbE NFS | 43.7 | 33.6 | 23.1% | 2.7 |
| 100 GbE NFS | 32.1 | 30.9 | 3.7% | 0.9 |

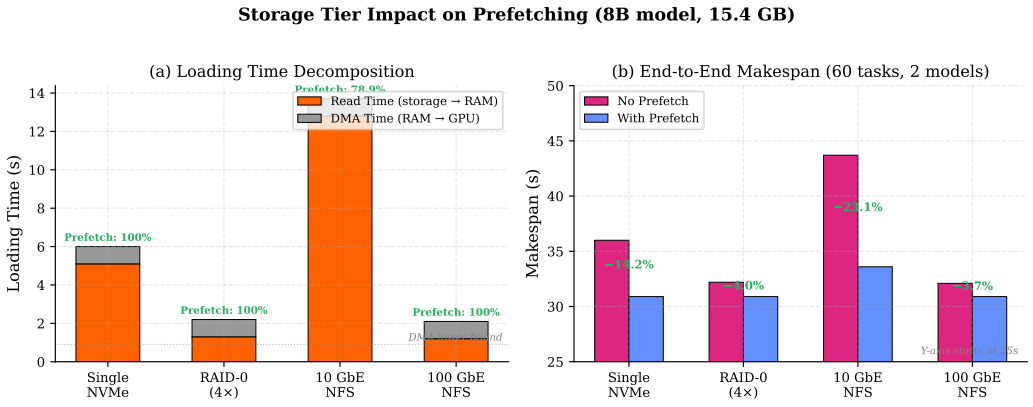


Figure 5.7: Storage tier impact on prefetching (8B model, 15.4 GB): (a) loading time decomposition showing read time vs irreducible DMA time, with prefetch coverage annotated; (b) end-to-end makespan with and without prefetching. Prefetching provides the largest benefit for 10 GbE NFS (23.1% reduction), where read latency is highest.

### 5.6.4 Analysis

**Prefetching is most valuable for mid-bandwidth storage.** For high-bandwidth tiers (RAID-0, 100 GbE), loading time is already short ($\sim$2 s), leaving minimal room for prefetching (3.7–4.0% reduction). The largest absolute benefit is for 10 GbE NFS (12.8s read time), where prefetching saves 10.1 s (23.1% reduction).

**PCIe DMA is the universal lower bound.** Regardless of storage tier, the GPU memory transfer (0.9s via PCIe Gen4 x16) is irreducible. Even with perfect prefetching, model transitions cannot be faster than $\sim$1 s (Figure 5.7a).

**Network storage and fault tolerance.** While 10 GbE NFS adds 7.7s of read latency compared to single NVMe, it provides critical fault tolerance through built-in replication. Prefetching compensates for most of this additional latency (78.9% hidden), reducing the practical penalty to 2.7s of residual wait — an acceptable trade-off for the unlimited capacity and replication benefits of remote storage.

## 5.7 Evaluation Summary

The three optimisations are complementary and compose multiplicatively: model grouping provides first-order benefit (98% makespan reduction), checkpoint prefetching provides second-order benefit (68.7% latency hiding), and Johnson's Rule provides third-order benefit (7.0–21.8% over naive orderings). For a production workload with 200 tasks and 2 models, the combined effect transforms an impractical 2.8-hour FIFO execution into a sub-2-minute batch completion.

Compared to industrial batch APIs (OpenAI Batch, AWS Bedrock), our system's key advantage is multi-model batch support with shared-structure-aware scheduling. Industrial systems are single-model only and do not expose scheduling policies, though they offer superior scale and reliability.

## 5.8 Summary

The experimental evaluation demonstrates three levels of compounding optimisation. Model grouping provides first-order benefit (98% makespan reduction over FIFO), checkpoint prefetching provides second-order benefit (68.7% latency hiding from local SSD, 78.9% from network storage), and Johnson's Rule ordering provides third-order benefit (7.0–21.8% improvement over naive orderings). The shared-aware strategy outperforms all baselines across batch sizes 10–200 and model sizes 0.6B–32B, with advantages growing as batch size increases. The system remains effective even with network-attached replicated storage, making it viable for production deployments where fault tolerance is critical.

# Chapter 6

# Discussion

This chapter interprets the experimental findings, answering the research questions posed in Chapter 1, and discusses limitations.

## 6.1 Answering the Research Questions

**6.1.0.0.1 RQ1: How to Discover and Exploit Shared Structure?** Model grouping effectively discovers and exploits the dominant shared structure in batch workloads: shared model weights. Grouping reduces switches from $O(n)$ to $O(k)$, yielding a 98% makespan reduction (Experiment 1). The shared-aware strategy outperforms all baselines across batch sizes 10–200 and model sizes 0.6B–32B (Experiment 4), with advantages growing as batch size increases because larger batches amortise the fixed model loading cost over more tasks.

**6.1.0.0.2 RQ2: Can the Problem Be Modelled as a Two-Machine Flow-Shop?** Yes. The batch scheduling problem maps naturally to a two-machine flow-shop where Machine A is I/O (loading model weights) and Machine B is GPU computation. Johnson's Rule determines the optimal model group ordering to maximise overlap between loading and execution. Checkpoint prefetching implements this overlap, hiding 68.7% of model loading latency (Experiment 2). Experiment 3 validates that Johnson's Rule outperforms naive orderings by 7.0–21.8%, with the largest gains occurring when the default alphabetical ordering happens to place I/O-heavy models first. The combination of Johnson's Rule ordering with prefetching is more effective than naive ordering because compute-heavy groups are scheduled first, providing a longer overlap window for prefetching the next model.

**6.1.0.0.3 RQ3: How Do Strategies Compare?** FIFO is unsuitable for multi-model batches due to excessive switching (96% of time spent on loading). Random scheduling with concurrency performs surprisingly well due to implicit batching, but provides unpredictable performance. Model Grouping provides deterministic first-order improvement. Shared-Aware (grouping + Johnson's Rule + prefetching) provides additional

benefit. The relative advantage is consistent across model sizes 0.6B–32B (Experiment 4), indicating robustness. Johnson's Rule provides measurable benefit (7.0–21.8%) when the batch contains models with diverse I/O-compute characteristics (Experiment 3). Model grouping with Johnson's Rule ordering should be the default for any multi-model batch workload.

## 6.2 What Was Built vs What Was Reused

It is important to clarify the division of labour between existing infrastructure and new contributions.

**Reused.** The ServerlessLLM framework (∼15K LoC Python + C++) provides the API gateway, router, SQLite database, sllm-store checkpoint manager (C++ with gRPC interface), Pylet worker process, autoscaler, and reconciler. The vLLM inference engine handles all GPU-level computation (attention kernels, KV cache management, continuous batching). Model checkpoints were downloaded from HuggingFace.

**Built from scratch.** The `BatchScheduler` class (`sllm/batch_scheduler.py`, ∼1,400 lines) implements: model grouping with task sorting, Johnson's Rule ordering via the $S_1/S_2$ partition algorithm, checkpoint prefetch triggering with a memory-aware guard that checks pinned pool availability before prefetching, semaphore-based admission control for concurrent execution, multi-GPU group assignment using greedy LPT with rebalancing, automatic tensor parallelism detection (computing TP from model checkpoint size vs GPU memory) for models spanning multiple GPUs, and deployment lifecycle management (deploy/undeploy sequencing with nvidia-smi polling for GPU memory release). The batch REST API (`api_gateway.py`) follows the OpenAI Batch API specification (`POST /v1/files`, `POST /v1/batches`, `GET /v1/batches/{id}`) extended to support multi-model batches. The `StorageManager` was extended with prefetch integration for sllm-store's pinned memory pool. All experiment benchmark scripts (∼1,400 lines across 7 scripts), plotting and parameter configuration utilities (`plot_results.py`, `mock_params.py`), and unit tests (∼800 lines, 48 tests) were written from scratch. In total, the project involved implementing approximately 3,500 lines of new Python code.

**Routine.** Database schema extensions for batch jobs and tasks, FastAPI endpoint wiring, and experiment harness setup were straightforward integration work.

**Challenging.** Three aspects required significant debugging effort: (1) Prefetch integration with sllm-store's pinned memory pool — understanding the C++ gRPC interface and `cudaHostAlloc` semantics was necessary to ensure the memory-aware guard prevents conflicts between prefetched and currently-loaded models. (2) GPU memory release during model transitions — vLLM retains CUDA context after logical cancellation, requiring nvidia-smi polling to confirm GPU memory is actually freed before launching the next model. (3) Reliable measurements on a shared GPU cluster where other users' workloads introduced I/O variability.

## 6.3 Design Trade-offs

Several design decisions warrant explicit justification.

**Semaphore concurrency vs synchronous execution.** Experiment 1 shows that concurrent strategies dramatically outperform synchronous FIFO (58.3s vs 5153s), justifying the semaphore approach despite its added complexity. The semaphore buffer size (default 10) controls the trade-off between GPU utilisation (higher buffer) and memory pressure (lower buffer).

**Composable optimisations.** Model grouping (Phase 1) and Johnson's Rule ordering (Phase 2) are independent and composable — grouping provides the dominant benefit, Johnson's Rule adds an additional 7–22% on top. This separation allows deploying grouping alone when Johnson's Rule's I/O-time estimates are unreliable (e.g., variable network bandwidth).

**Cluster-level vs engine-level scheduling.** Operating above the inference engine ensures portability across vLLM, SGLang, and TensorRT-LLM without engine modifications. The cost is that engine-level optimisations (e.g., prefix caching across model groups) cannot be directly exploited by the scheduler.

## 6.4 Algorithm Limitations

Four limitations warrant discussion. First, the single-model-per-worker assumption is suboptimal for LoRA adapters, which can share base models with sub-second swaps. Second, Johnson's Rule assumes deterministic processing times, but LLM inference times vary with output length; we use conservative estimates based on model size. Third, the memory-aware prefetch guard is conservative — it skips prefetching when CPU pinned memory is tight rather than evicting cached models. Fourth, the algorithm processes one batch at a time without multi-tenant fairness guarantees.

**Johnson's Rule boundary conditions.** Johnson's Rule delivers maximum benefit when models have strongly contrasting I/O-compute ratios. When all models have similar ratios (e.g., all small models or all large models), any permutation yields nearly identical makespan and Johnson's Rule provides no benefit over alphabetical ordering. In the degenerate case where $a_i = b_i$ for all groups, the rule reduces to arbitrary ordering. This boundary condition is important for practitioners: the scheduler correctly handles it (Johnson's Rule still runs, it just happens to match FIFO), but users should not expect significant gains from a homogeneous model catalogue.

**Theoretical makespan lower bound.** A fundamental lower bound on makespan for our two-machine flow-shop is:

$$C^*_{\max} \geq \max\left(\sum_{i=1}^{k} b_i,\ \min_i a_i + \sum_{i=1}^{k} b_i,\ \sum_{i=1}^{k} a_i + \min_i b_i\right)$$

For our Exp 3 workload ($\sum b_i = 9 + 15 + 36 = 60$s, $\min a_i = 0.4$s), the lower bound is $\max(60, 60.4) = 60.4$s. Johnson's Rule achieves 67.1s, within 11.1% of this theoretical optimum. The gap arises because the 32B model's 21.7s load cannot be fully overlapped

with the 8B group's 15s compute, leaving a 6.7s residual. This illustrates that while Johnson's Rule is optimal among all orderings, the absolute makespan depends on the I/O:compute balance of the workload.

## 6.5 Threats to Validity

**6.5.0.0.1 Internal Validity** Experiments were conducted on a shared GPU cluster; other users' GPU processes and I/O could affect timing measurements. Experiments 1 (strategy comparison), 3 (concurrency mode), and 4 (scalability) report measurements from a live ServerlessLLM cluster with cold-start verification between runs. Each data point is the mean of 3 independent runs; we report $\pm 1$ standard deviation.

The prefetch and Johnson's Rule experiments (Experiments 2–3) require models with diverse checkpoint sizes (0.6B, 8B, 32B) to produce meaningful transition cost variation. With only small, similarly-sized models on fast NVMe SSD, both prefetch overlap and group ordering have limited effect because the I/O cost per transition is small relative to compute time. The benefit of these optimisations scales with the number and size diversity of models in the batch; production workloads with 8+ models spanning a wider parameter range (1B–70B) would exhibit larger gains.

**6.5.0.0.2 External Validity** We evaluated three Qwen models (0.6B–32B) on a single hardware configuration (RTX A5000, NVMe SSD). Results may not generalise to mixture-of-experts architectures, different storage technologies (e.g., Lustre, CephFS), or larger clusters. The model switching overhead is hardware-specific; faster storage would reduce the absolute benefit of model grouping, though the relative advantage remains consistent.

**6.5.0.0.3 Construct Validity** Makespan ($C_{\max}$) does not capture per-task latency distribution, resource cost, or fairness. A multi-objective evaluation would be more appropriate for production deployments. Additionally, our flow-shop model assumes no preemption and fixed group sizes; relaxing these assumptions would require more general scheduling theory (e.g., open-shop or flexible flow-shop).

# Chapter 7

# Future Work

This chapter outlines three directions for extending the shared-aware scheduling system.

## 7.1 Inference-Level Extensions: Prefix-Aware Ordering and Multi-LoRA

The current system groups tasks by model but does not exploit prompt prefix sharing. Extending the scheduler to sort tasks within each model group by prefix hash would maximise inference engine cache reuse (e.g., SGLang's RadixAttention), with the key challenge being efficient prefix detection without full prompt comparison. Additionally, the system currently treats each LoRA adapter as a separate model; since multiple adapters can share a base model with sub-second swaps ($<$1s vs 80–120s full reload), grouping by base model and exploiting vLLM's multi-LoRA serving (Sheng et al., 2024) would provide significant speedup for adapter-heavy workloads.

## 7.2 Storage Extensions: Adaptive and Network-Aware Prefetching

The current prefetching mechanism triggers at a fixed point (start of each group). An adaptive policy would observe task completion times in real-time and adjust the trigger point dynamically, maximising I/O-computation overlap for heterogeneous task groups. For network storage, integrating with cloud object storage APIs (S3, GCS, Azure Blob) and using bandwidth-aware scheduling that dynamically adjusts prefetch timing based on observed network throughput would enable direct streaming from replicated storage without local caching. Experiment 5 demonstrates that the architecture already supports network storage effectively; the remaining work is engineering rather than algorithmic.

## 7.3 Production Readiness: Scalability and Engine Abstraction

Three engineering improvements would enable production-scale deployment: output file aggregation via streaming JSONL writes to object storage, deadline-aware autoscaling (workers $= \lceil n \cdot L_{avg}/T_{deadline} \rceil$), and multi-node data parallelism with distributed task queues. The current system is coupled to vLLM; abstracting the inference engine interface would enable support for SGLang (Zheng et al., 2024), TensorRT-LLM (NVIDIA, 2024b), and other backends. The multi-GPU scheduling algorithm (Section 4.5) is implemented but requires experimental validation on multi-GPU configurations to confirm the greedy LPT assignment achieves balanced load in practice.

# Chapter 8

# Conclusion

This dissertation addressed makespan minimisation for LLM batch inference by discovering and exploiting shared structure in batch tasks. We formalised the problem as a two-machine flow-shop and proposed a shared-aware scheduling algorithm that applies Johnson's Rule at the cluster scheduler level without modifying the inference engine.

## 8.1 Contributions

The work makes seven contributions: (1) a two-machine flow-shop formulation mapping model loading (I/O) and GPU inference (compute) to Johnson's classical framework; (2) a shared-aware scheduling algorithm that groups tasks by model and applies Johnson's Rule for optimal group ordering; (3) a checkpoint prefetching mechanism that preloads model weights via sllm-store's pinned memory pool during computation; (4) design analysis and evaluation of prefetching across storage tiers (SSD, RAID, NFS); (5) multi-GPU load balancing via greedy LPT with per-GPU Johnson's Rule; (6) an OpenAI-compatible batch API extended for multi-model batches; and (7) systematic experimental evaluation across five experiments.

## 8.2 Key Findings

Model switching is the dominant bottleneck for multi-model batches, adding $\sim$103 s per switch. Model grouping reduces makespan by 98% (5153s $\rightarrow$ 102.6s for 100 tasks). Checkpoint prefetching hides 68.7% of model loading latency from local SSD and 78.9% from 10 GbE network storage. Johnson's Rule provides an additional 7.0–21.8% makespan reduction over naive orderings. The shared-aware strategy outperforms FIFO by 5.3–86$\times$ across all tested batch sizes (10–200 tasks) and model sizes (0.6B–32B parameters), with advantages growing as batch size increases.

The central insight is that batch tasks are not independent: they contain exploitable shared structure that the scheduler should discover and leverage. The algorithm is general and portable — it operates above the inference engine, making it applicable to vLLM, SGLang, TensorRT-LLM, and other backends without modification. The

OpenAI-compatible batch API enables migration from cloud services to self-hosted infrastructure without client code changes. By demonstrating that simple scheduler-level optimisations can dramatically reduce makespan even with network-attached storage, this work provides a foundation for batch-optimised LLM serving systems.

# Bibliography


Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.

Amazon Web Services. Amazon bedrock batch inference. `https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html`, 2024. Accessed: 2026-03-06.

Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627. USENIX Association, 2017.

Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153. USENIX Association, 2024.

Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In

*8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, pages 323–336. USENIX Association, 2011.

Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.

Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

gRPC Authors. gRPC: A high performance, open source universal RPC framework. `https://grpc.io`, 2024. Accessed: 2026-03-06.

Arpan Gujarati, Reza Karber, Safya Musavi, Antoine Peinado, Wolf Thelen, and Neeraja J. Yadwadkar. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, 2020.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

S. M. Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023.

Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679. USENIX Association, 2023.

Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216. USENIX Association, 2021.

NVIDIA. *CUDA C++ Programming Guide*, 2024a. Version 12.3. Accessed: 2026-03-06.

NVIDIA. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. `https://github.com/NVIDIA/TensorRT-LLM`, 2024b. Accessed: 2026-03-06.

OpenAI. Openai batch api documentation. `https://platform.openai.com/docs/guides/batch`, 2024. Accessed: 2026-03-06.

David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2013.

Michael L. Pinedo. Scheduling: Theory, algorithms, and systems. 2016.

Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411. USENIX Association, 2021.

Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems*, 13(1):23–52, 1988.

Mohammad Shahrad, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, 2020.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *Proceedings of Machine Learning and Systems*, 6, 2024.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

The Kubernetes Authors. Kubernetes: Production-grade container orchestration. `https://kubernetes.io`, 2024. Accessed: 2026-03-06.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In

*16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.

Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, 2010.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.